

VALUE STRATEGY CONSULTING

Documentacion Tecnica

OCP Maintenance AI Platform - Sistema Empresarial de Gestion de Mantenimiento

Campo	Detalle
Documento	DOC-OCP-2026-001
Fecha	Marzo 2026
Version	1.0 - Final
Clasificacion	Confidencial - Uso Interno
Alcance	Codigo fuente completo, infraestructura Docker, API
Metodologia	Revision manual de codigo + analisis estatico
Sistema	FastAPI + React 19 + PostgreSQL + Docker
Evaluated por	Value Strategy Consulting (VSC)

OCP Maintenance AI Platform

Built for reliability, designed to scale.

Tabla de Contenidos

1.	Descripcion General
2.	Arquitectura del Sistema
3.	Stack Tecnologico
4.	Estructura del Proyecto
5.	Backend (FastAPI)
6.	Frontend (React + Vite)
7.	Esquema de Base de Datos.....
8.	Autenticacion y Autorizacion
9.	Referencia de API
10.	Internacionalizacion (i18n)
11.	Integracion SAP
12.	Docker y Despliegue
13.	Modulos Funcionales
14.	Flujos de Trabajo Clave
15.	Seguridad
16.	Guia de Desarrollo Local
17.	Checklist de Produccion
18.	Variables de Entorno

Version: 1.0.0
Ultima Actualizacion: Febrero 2026
Plataforma: Aplicacion Web Full-Stack Contenerizada
Cliente: OCP Group (Office Cherifien des Phosphates)

1. Descripcion General

1.1 Que es este proyecto?

OCP Maintenance AI Platform es un sistema empresarial de gestion de mantenimiento con 4 modulos, potenciado por inteligencia artificial. Fue disenado para OCP Group, el mayor exportador de fosfatos del mundo, para optimizar las operaciones de mantenimiento en su complejo industrial Jorf Lasfar (JFC1).

1.2 Objetivos Principales

Objetivo	Descripcion
Mantenimiento Predictivo	Usar analisis Weibull e IA para predecir fallas de equipos antes de que ocurran
Optimizacion RCM	Aplicar metodologia de Mantenimiento Centrado en Confiabilidad para optimizar estrategias
Integracion SAP	Generar ordenes de trabajo, listas de tareas y paquetes de carga compatibles con SAP
Soporte Multi-Idioma	Interfaz completa en Ingles, Espanol y Arabe (RTL)
Control de Acceso por Roles	5 roles de usuario con control granular de permisos
Analitica en Tiempo Real	KPIs, puntuaciones de salud, OEE, tableros MTBF/MTTR

1.3 Los 4 Modulos

Modulo 1: JERARQUIA DE ACTIVOS Y CRITICIDAD
Gestion del arbol de equipos, evaluacion de matriz de riesgo

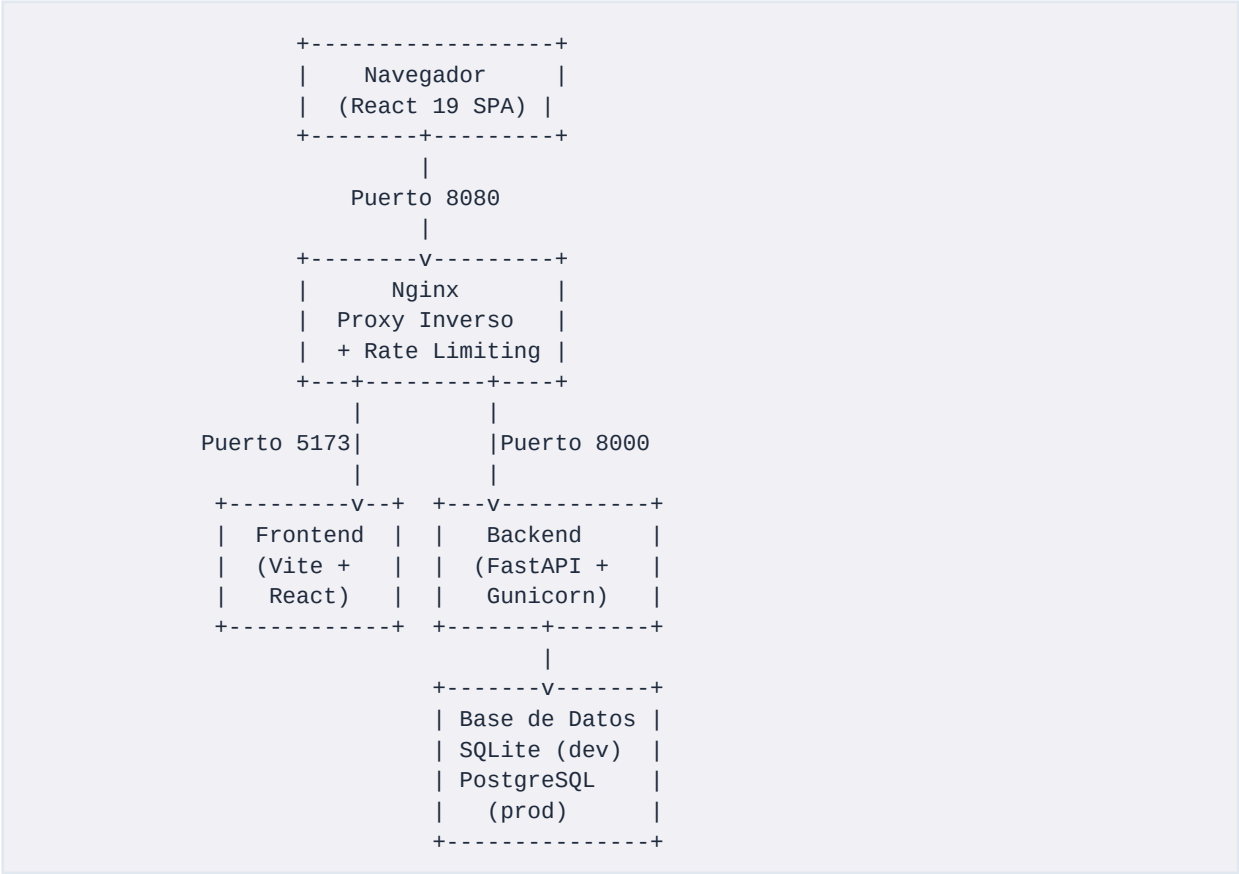
Modulo 2: FMEA / FMECA Y ESTRATEGIA RCM
Analisis de modos de falla, logica de decision RCM, generacion de tareas

Modulo 3: OPERACIONES Y PLANIFICACION
Solicitudes de trabajo, gestion de backlog, programacion, paquetes de trabajo

Modulo 4: ANALITICA, REPORTES Y MEJORA CONTINUA
KPIs, analisis Weibull, RCA (5W2H), eliminacion de defectos, exportacion SAP

2. Arquitectura del Sistema

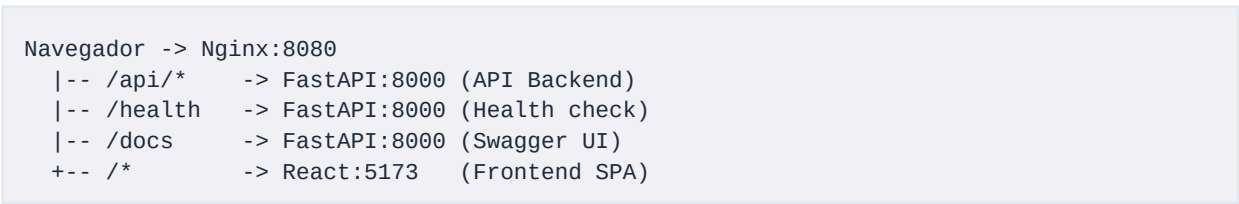
2.1 Diagrama General



2.2 Puertos de los Servicios

Servicio	Puerto	Descripcion
Frontend (Vite)	5173	Servidor de desarrollo React
Backend (FastAPI)	8000	API + Documentacion Swagger en /docs
Nginx (Proxy)	8080	Punto de entrada unificado
Nginx (SSL)	8443	HTTPS (produccion)

2.3 Flujo de Peticiones



3. Stack Tecnológico

3.1 Backend

Tecnologia	Version	Proposito
Python	3.11	Runtime
FastAPI	0.133.1	Framework web
Uvicorn	0.41.0	Servidor ASGI
Gunicorn	25.1.0	Gestor de procesos (produccion)
SQLAlchemy	2.0.47	ORM
Pydantic	2.12.5	Validacion de datos
python-jose	3.3.0	Tokens JWT
bcrypt	4.2.1	Hash de contraseñas
Anthropic SDK	0.83.0	Integracion con Claude AI
openpyxl	3.1.5	Generacion de Excel
httpx	0.28.1	Cliente HTTP

3.2 Frontend

Tecnologia	Version	Proposito
React	19.1.0	Framework UI
Vite	7.3.1	Herramienta de build y servidor dev
Tailwind CSS	4.2.1	Estilos utility-first
React Router DOM	7.13.1	Enrutamiento del lado del cliente
Radix UI	Ultimo	Componentes accesibles headless
Recharts	3.7.0	Visualizacion de datos / graficos
Lucide React	0.575.0	Libreria de iconos
XLSX	0.18.5	Exportacion Excel del lado del cliente
file-saver	2.0.5	Utilidad de descarga de archivos

3.3 Infraestructura

Tecnologia	Proposito
Docker	Contenerizacion
Docker Compose	Orquestacion multi-servicio

4. Estructura del Proyecto

Nginx Alpine	Proxy inverso + servicio de archivos estaticos
SQLite	Base de datos de desarrollo
PostgreSQL	Base de datos de produccion

```

ASSET-MANAGEMENT-SOFTWARE-master/
|
|-- api/                                # === BACKEND (FastAPI) ===
|   |-- main.py                         # Entrada: middleware, routers, health
|   |-- config.py                      # Configuración desde .env
|   |-- schemas.py                    # 50+ modelos Pydantic request/response
|   |-- seed.py                       # Script de semilla de base de datos
|   |
|   |-- database/
|   |   |-- connection.py             # Motor SQLAlchemy y sesiones
|   |   +-- models.py                # 25+ modelos ORM
|   |
|   |-- routers/
|   |   |-- admin.py                 # Módulos de endpoints API
|   |   |-- analytics.py             # Semilla, estadísticas, auditoría
|   |   |-- auth.py                 # Puntuaciones de salud, KPIs, Weibull
|   |   |-- backlog.py              # Login, registro, JWT
|   |   |-- capture.py              # Gestión de backlog
|   |   |-- criticality.py          # Captura de datos en campo
|   |   |-- dashboard.py            # Evaluación de riesgo
|   |   |-- fmea.py                 # Tablero ejecutivo
|   |   |-- hierarchy.py            # FMEA/FMECA, decisiones RCM
|   |   |-- planner.py              # CRUD árbol de equipos
|   |   |-- rca.py                  # Recomendaciones del planificador IA
|   |   |-- reliability.py           # Análisis de Causa Raíz
|   |   |-- reporting.py            # Repuestos, paradas, MOCs
|   |   |-- sap.py                  # Reportes, exportaciones, notificaciones
|   |   |-- scheduling.py           # Integración SAP
|   |   |-- tasks.py                # Programación, GANTT
|   |   |-- work_packages.py        # CRUD tareas de mantenimiento
|   |   +-- work_requests.py        # Agrupación de paquetes de trabajo
|   |
|   |-- services/
|   |   |-- agent_service.py         # Capa de lógica de negocio (23 servicios)
|   |   |-- analytics_service.py     # Configuración del agente IA
|   |   |-- audit_service.py        # Cálculos de KPI
|   |   |-- auth_service.py         # Registro de auditoría
|   |   |-- backlog_service.py      # JWT, hash de contraseñas
|   |   |-- criticality_service.py  # Optimización de backlog
|   |   |-- fmea_service.py         # Motor de matriz de riesgo
|   |   |-- hierarchy_service.py    # Lógica FMEA + RCM
|   |   |-- planner_service.py      # Jerarquía de equipos
|   |   |-- rca_service.py          # Algoritmos de planificación
|   |   |-- reliability_service.py  # Análisis 5W2H
|   |   |-- reporting_service.py    # Predicción de fallas
|   |   |-- sap_service.py          # Generación de reportes
|   |   |-- scheduling_service.py   # Procesamiento de datos SAP
|   |   |-- work_package_service.py # Optimización de programación
|   |   |-- work_request_service.py # Lógica de agrupación WP
|   |   +-- ...                    # Procesamiento de WR
|   |
|   +-- dependencies/
|   |   +-- auth.py                # Servicios adicionales
|   |
|-- frontend/                          # === FRONTEND (React SPA) ===
|   |-- package.json                 # Dependencias npm
|   |-- vite.config.js               # Configuración Vite + proxy
|   |-- index.html                   # Entrada SPA (favicon, meta)
|   |-- Dockerfile                   # Build multi-etapa
|

```

```

|   +-- src/
|   |   |-- main.jsx           # Configuracion de rutas (23 rutas)
|   |   |-- App.jsx           # Layout raiz (Sidebar + Header)
|   |   |-- api.js            # Cliente API (50+ endpoints)
|   |
|   |   |-- pages/            # 23 componentes de pagina
|   |   |   |-- Login.jsx     # Autenticacion
|   |   |   |-- Dashboard.jsx # Vista general de KPIs
|   |   |   |-- Hierarchy.jsx # Arbol de equipos
|   |   |   |-- Criticality.jsx # Evaluacion de riesgo
|   |   |   |-- FMEA.jsx      # Analisis de modos de falla
|   |   |   |-- FMECA.jsx     # FMEA con criticidad
|   |   |   |-- Strategy.jsx  # Seleccion de estrategia RCM
|   |   |   |-- WorkRequests.jsx # Gestion de solicitudes
|   |   |   |-- WorkPackages.jsx # Agrupacion de tareas + exportacion SAP
|   |   |   |-- Backlog.jsx   # Backlog de mantenimiento
|   |   |   |-- Scheduling.jsx # Programacion
|   |   |   |-- Planning.jsx  # Planificacion a largo plazo
|   |   |   |-- Planner.jsx   # Chat asistente IA
|   |   |   |-- FieldCapture.jsx # Captura de datos movil
|   |   |   |-- Analytics.jsx  # Tableros de KPI
|   |   |   |-- ExecutiveDashboard.jsx # Resumen ejecutivo
|   |   |   |-- Reports.jsx    # Generacion de reportes + exportacion
|   |   |   |-- Reliability.jsx # Repuestos, paradas
|   |   |   |-- RCA.jsx        # Analisis de Causa Raiz
|   |   |   |-- DefectElimination.jsx # Seguimiento de defectos
|   |   |   |-- SAPReview.jsx  # Revision de cargas SAP
|   |   |   |-- Admin.jsx      # Gestion de usuarios + configuracion
|   |   |   +-- Profile.jsx    # Perfil del usuario
|   |
|   |   |-- components/      # Componentes UI reutilizables
|   |   |   |-- Sidebar.jsx   # Menu de navegacion
|   |   |   |-- Header.jsx    # Barra superior + selector de planta
|   |   |   |-- ProtectedRoute.jsx # Guardia de acceso por roles
|   |   |   |-- ErrorBoundary.jsx # Manejo de errores
|   |   |   |-- Toast.jsx     # Sistema de notificaciones
|   |   |   |-- ConfirmDialog.jsx # Dialogos de confirmacion
|   |   |   |-- Shared.jsx    # StatusBadge, LoadingSpinner
|   |   |   +-- ui/          # 15+ primitivas Radix UI
|   |
|   |   |-- contexts/        # Estado global
|   |   |   |-- AuthContext.jsx # Estado de autenticacion
|   |   |   +-- LanguageContext.jsx # Seleccion de idioma i18n
|   |
|   |   |-- i18n/            # Traducciones
|   |   |   |-- en.js        # Ingles
|   |   |   |-- es.js        # Espanol
|   |   |   +-- ar.js        # Arabe (RTL)
|   |
|   |   |-- data/
|   |   |   +-- mockData.js   # Datos mock de desarrollo
|   |
|   |   |-- utils/           # Funciones utilitarias
|   |   |   +-- exportFile.js # Helper de descarga Excel/CSV
|   |
|   |   +-- styles/
|   |       +-- index.css     # CSS global + modo oscuro
|
|-- sap_mock/                # === SERVICIO MOCK SAP ===
|   |-- generate_mock_data.py # Generador de datos mock

```



```
|   +-- data/                                # Datasets JSON
|       |-- work_orders.json                 # Ordenes de trabajo historicas
|       |-- functional_locations.json        # Ubicaciones de equipos
|       |-- materials_bom.json               # Lista de materiales
|       |-- equipment_master.json            # Catalogo de equipos
|       +-- maintenance_plans.json           # Planes predefinidos
|
|-- Dockerfile                               # Contenedor del backend
|-- docker-compose.yml                       # Orquestacion de 3 servicios
|-- nginx.conf                               # Configuracion del proxy inverso
|-- requirements.txt                          # Dependencias Python (27 paquetes)
|-- .env.example                             # Plantilla de variables de entorno
+-- ocp_maintenance.db                       # Base de datos SQLite (dev)
```

5. Backend (FastAPI)

5.1 Inicializacion de la Aplicacion

La aplicacion se crea en `api/main.py` mediante la fabrica `create_app()`:

1. Instancia FastAPI con Swagger UI en `/docs`
2. Middleware CORS · permite origenes del frontend desde `.env`
3. Middleware de API Key · aplica X-API-Key en mutaciones cuando esta configurado
4. 18 routers registrados bajo el prefijo `/api/v1`
5. Tablas de base de datos creadas al inicio mediante el manejador `lifespan`
6. Endpoint de salud en `/health` con verificacion de conectividad DB

5.2 Routers (18 Modulos)

Router	Prefijo	Proposito
auth	/auth	Login, registro, tokens JWT, gestion de usuarios
hierarchy	/hierarchy	CRUD de plantas y nodos de equipos
criticality	/criticality	Evaluacion de matriz de riesgo y aprobacion
fmea	/fmea	Funciones, fallas, modos de falla, RCM, FMECA
tasks	/tasks	CRUD de tareas de mantenimiento
work_packages	/work-packages	Agrupacion de tareas, aprobacion, instrucciones
sap	/sap	Generacion de cargas SAP, acceso a datos mock
analytics	/analytics	Puntuaciones de salud, KPIs, analisis Weibull
work_requests	/work-requests	Validacion de WR, duplicados, OCR
capture	/capture	Captura de datos en campo
planner	/planner	Recomendaciones del planificador IA
backlog	/backlog	Gestion y optimizacion de backlog
scheduling	/scheduling	Programacion, GANTT
reliability	/reliability	Repuestos, paradas, MOCs
reporting	/reporting	Reportes, notificaciones, exportacion
dashboard	/dashboard	KPIs ejecutivos y alertas
rca	/rca	Analisis de Causa Raiz (5W2H)
admin	/admin	Semilla, estadisticas, auditoria, feedback

5.3 Capa de Servicios

Cada router delega la logica de negocio a un servicio correspondiente en `api/services/`. Servicios clave:

- `auth_service.py` · Hash de contraseñas (bcrypt), creacion/validacion JWT, CRUD de usuarios

- fmea_service.py · Logica FMEA/FMECA, arboles de decision RCM, calculo RPN
- analytics_service.py · Ajuste Weibull, computo de puntuacion de salud, agregacion KPI
- criticality_service.py · Evaluacion de matriz de riesgo, clases de riesgo sugeridas por IA
- work_package_service.py · Algoritmos de agrupacion de tareas, generacion de cargas SAP
- reporting_service.py · Generacion de reportes semanales/mensuales, exportacion Excel
- rca_service.py · Framework de analisis de causa raiz 5W2H
- capture_service.py · Procesamiento de capturas de campo con deteccion de modos de falla

5.4 Configuracion

api/config.py carga configuraciones desde variables de entorno:

```
DATABASE_URL      # Cadena de conexion SQLite o PostgreSQL
SAP MOCK DIR      # Ruta al directorio de datos mock SAP
JWT_SECRET_KEY    # Secreto para firmar tokens JWT
JWT_ALGORITHM     # HS256 (por defecto)
CORS_ORIGINS      # Origenes permitidos separados por coma
API_V1_PREFIX     # /api/v1 (por defecto)
LOG_LEVEL         # INFO, DEBUG, WARNING, etc.
```

6. Frontend (React + Vite)

6.1 Entrada de la Aplicacion

main.jsx configura React Router con 23 rutas lazy-loaded envueltas en guardias basadas en roles:

/login	-> Login.jsx	(publico)
/	-> Dashboard.jsx	(todos los roles)
/hierarchy	-> Hierarchy.jsx	(todos los roles)
/criticality	-> Criticality.jsx	(todos los roles)
/fmea	-> FMEA.jsx	(planner, engineer, admin)
/fmeca	-> FMECA.jsx	(planner, engineer, admin)
/strategy	-> Strategy.jsx	(planner, engineer, admin)
/work-requests	-> WorkRequests.jsx	(todos los roles)
/work-packages	-> WorkPackages.jsx	(planner, admin)
/backlog	-> Backlog.jsx	(planner, admin)
/scheduling	-> Scheduling.jsx	(planner, admin)
/planner	-> Planner.jsx	(planner, admin)
/planning	-> Planning.jsx	(planner, admin)
/field-capture	-> FieldCapture.jsx	(todos los roles)
/analytics	-> Analytics.jsx	(manager, admin)
/executive	-> ExecutiveDashboard	(manager, admin)
/reports	-> Reports.jsx	(manager, admin)
/reliability	-> Reliability.jsx	(engineer, admin)
/rca	-> RCA.jsx	(engineer, admin)
/defect-elimination	-> DefectElimination	(engineer, admin)
/sap-review	-> SAPReview.jsx	(manager, admin)
/admin	-> Admin.jsx	(solo admin)
/profile	-> Profile.jsx	(todos los roles)

6.2 Sistema de Layout

App.jsx proporciona el layout principal:

- Sidebar · Navegacion verde (#1B5E20) con logo OCP, items de menu filtrados por rol, colapsable
- Header · Selector de planta, selector de idioma (EN/ES/AR), modo oscuro, menu de usuario
- Area de Contenido · Renderiza la pagina activa via <Outlet />

6.3 Gestion de Estado

Contexto	Archivo	Proposito
AuthContext	contexts/AuthContext.jsx	Sesion de usuario, tokens JWT, login/logout, verificacion de roles
LanguageContext	contexts/LanguageContext.jsx	Idioma activo, funcion traductora t(), deteccion RTL

6.4 Cliente API

api.js proporciona 50+ funciones que envuelven llamadas HTTP al backend:

- URL base: /api/v1 (proxy Vite en dev, Nginx en prod)
- Header automatico Authorization: Bearer {token}
- Refresco automatico de token ante respuestas 401
- Manejo centralizado de errores

Ejemplo:

```
export const listWorkRequests = (params) => get('/work-requests', params);
export const validateWorkRequest = (id, d) => put(`/work-requests/${id}/validate`, d);
export const calculateHealthScore = (d) => post('/analytics/health-score', d);
```

6.5 Libreria de Componentes UI

Construida sobre Radix UI (headless, accesible) + Tailwind CSS (utility-first):

Componente	Descripcion
Sidebar.jsx	Navegacion principal con filtrado por roles
Header.jsx	Barra superior con selectores de planta/idioma/tema
ProtectedRoute.jsx	Guardia de ruta que verifica roles de usuario
Toast.jsx	Toasts de notificacion exito/error/info
ConfirmDialog.jsx	Dialogos modales de confirmacion
Shared.jsx	StatusBadge, LoadingSpinner, DataTable
ui/*.jsx	15+ primitivas Radix (Button, Dialog, Select, Tabs, etc.)

6.6 Tematizacion

- Modo claro/oscuro via propiedades CSS personalizadas (--background, --foreground, etc.)
 - Color de marca OCP: #1B5E20 (verde oscuro) como primario
 - Sidebar: Fondo verde solido bg-[#1B5E20]
 - Tarjetas: bg-card border-border adaptativo al tema
 - Badges de estado: Codificados por color (verde=aprobado, ambar=borrador, azul=revision, rojo=critico)
-

7. Esquema de Base de Datos

7.1 Modelos Principales (25+ Tablas)

```
Usuarios y Autenticacion
|-- UserModel          # Usuarios con roles (admin, manager, planner, tecnico)

Jerarquia de Activos
|-- PlantModel         # Instalaciones de manufactura
|-- HierarchyNodeModel # Arbol de equipos (6 niveles de profundidad)

Criticidad
|-- CriticalityAssessmentModel # Evaluaciones de matriz de riesgo con flujo de aprobacion

FMEA / FMECA
|-- FunctionModel      # Funciones del equipo
|-- FunctionalFailureModel # Fallas funcionales (TOTAL/PARCIAL)
|-- FailureModeModel   # Modos de falla con mecanismo/causa
|-- MaintenanceTaskModel # Tareas derivadas de RCM (CBM/TBM/FTM/RTF)

Gestion de Trabajo
|-- WorkPackageModel   # Tareas de mantenimiento agrupadas
|-- WorkOrderModel     # Ordenes de trabajo historicas SAP

Analitica
|-- HealthScoreModel   # Salud del activo multidimensional
|-- KPIMetricsModel    # MTBF, MTTR, disponibilidad, OEE
|-- FailurePredictionModel # Predicciones basadas en Weibull

Integracion SAP
|-- SAPUploadPackageModel # Paquetes de exportacion SAP

Mejora Continua
|-- CAPAItemModel      # Acciones correctivas/preventivas
```

7.2 Niveles de Jerarquia de Equipos

```
PLANTA (ej., OCP-JFC1)
+-- AREA (ej., Molienda)
    +-- SISTEMA (ej., Circuito Molino SAG)
        +-- EQUIPO (ej., Molino SAG #1)
            +-- SUB_ENSAMBLE (ej., Conjunto de Accionamiento)
                +-- ITEM_MANTENIBLE (ej., Rodamiento Principal)
```

8. Autenticacion y Autorizacion

8.1 Flujo de Tokens JWT

1. POST /auth/login { username, password }
-> Retorna: { access_token (4h), refresh_token (7d), user }
2. Todas las peticiones API incluyen:
Authorization: Bearer <access_token>
3. Ante 401 (token expirado):
POST /auth/refresh { refresh_token }
-> Retorna: nuevo access_token + refresh_token
4. Si el refresco falla:
-> Redireccion a /login

8.2 Roles de Usuario y Permisos

Rol	Acceso
admin	Acceso total al sistema + gestion de usuarios
manager	Supervision de operaciones, analitica, reportes, revision SAP
planner	FMEA, paquetes de trabajo, backlog, programacion, planificador IA
tecnico	Dashboard, captura en campo, solicitudes de trabajo, jerarquia
engineer	Confiabilidad, RCA, eliminacion de defectos, FMEA

8.3 Usuarios Demo por Defecto

Usuario	Contraseña	Rol
admin	admin123	admin
manager	manager123	manager
planner	planner123	planner
tecnico	tecnico123	tecnico

8.4 Seguridad de Contraseñas

- Hash con bcrypt (rondas de sal)
 - Nunca almacenadas en texto plano
 - Validacion minima en el registro
 - Cambio de contraseña requiere verificacion de la contraseña anterior
-

9. Referencia de API

9.1 URL Base

Desarrollo: <http://localhost:8000/api/v1>
Produccion: <https://tu-dominio.com/api/v1>
Swagger UI: <http://localhost:8000/docs>

9.2 Resumen de Endpoints

Autenticacion (`/auth`)

Metodo	Endpoint	Descripcion
POST	/auth/login	Iniciar sesion con credenciales
POST	/auth/refresh	Refrescar tokens JWT
POST	/auth/register	Crear nuevo usuario (admin)
GET	/auth/me	Perfil del usuario actual
PUT	/auth/me	Actualizar perfil
PUT	/auth/change-password	Cambiar contrasena
GET	/auth/users	Listar todos los usuarios (admin)
PUT	/auth/users/{id}/role	Actualizar rol de usuario
PUT	/auth/users/{id}/activate	Activar usuario
PUT	/auth/users/{id}/deactivate	Desactivar usuario

Jerarquia (`/hierarchy`)

Metodo	Endpoint	Descripcion
GET	/hierarchy/plants	Listar plantas
POST	/hierarchy/plants	Crear planta
GET	/hierarchy/nodes	Listar nodos de equipos
POST	/hierarchy/nodes	Crear nodo
GET	/hierarchy/nodes/{id}	Obtener detalles del nodo
GET	/hierarchy/nodes/{id}/tree	Obtener subarbol
POST	/hierarchy/build-from-vendor	Importar desde datos del proveedor
GET	/hierarchy/stats	Estadisticas de jerarquia

FMEA (`/fmea`)

Metodo	Endpoint	Descripcion
POST	/fmea/functions	Crear funcion del equipo
GET	/fmea/functions	Listar funciones
POST	/fmea/functional-failures	Crear falla funcional
POST	/fmea/failure-modes	Crear modo de falla
GET	/fmea/failure-modes	Listar modos de falla
GET	/fmea/validate-fm	Validar combinacion FM
GET	/fmea/fm-combinations	Pares mecanismo/causa validos
POST	/fmea/rcm-decide	Decision de estrategia RCM
POST	/fmea/fmeca/worksheets	Crear hoja de trabajo FMECA
POST	/fmea/fmeca/rpn	Calcular RPN

Analitica (`/analytics`)

Metodo	Endpoint	Descripcion
POST	/analytics/health-score	Calcular salud del activo
POST	/analytics/kpis	Calcular metricas KPI
POST	/analytics/weibull-fit	Ajustar distribucion Weibull
POST	/analytics/weibull-predict	Predecir proxima falla
GET	/analytics/variance-alerts	Anomalias estadisticas
GET	/analytics/asset-health	Resumen de salud

Reportes (`/reporting`)

Metodo	Endpoint	Descripcion
POST	/reporting/reports/weekly	Generar reporte semanal
POST	/reporting/reports/monthly	Generar reporte mensual
POST	/reporting/reports/quarterly	Generar reporte trimestral
GET	/reporting/reports	Listar reportes
POST	/reporting/export	Exportar datos (Excel/CSV/SAP)
GET	/reporting/notifications	Listar notificaciones
POST	/reporting/cross-module/analyze	Analisis cross-modulo

10. Internacionalizacion (i18n)

10.1 Idiomas Soportados

Idioma	Codigo	Direccion	Archivo
Inglés	en	LTR	frontend/src/i18n/en.js
Español	es	LTR	frontend/src/i18n/es.js
Árabe	ar	RTL	frontend/src/i18n/ar.js

10.2 Implementacion

El sistema i18n usa React Context (LanguageContext.jsx):

```
// Acceder a traducciones en cualquier componente:
const { t, lang, setLang } = useLanguage();

// Uso:
t('nav.dashboard') // -> "Panel Principal"
t('common.noResults', { query: 'bomba' }) // -> "Sin resultados para 'bomba'"
t('planner.suggestions') // -> ["Sugerencia 1", "Sugerencia 2", ...]
```

10.3 Estructura de Traducciones

```
{
  common: { // Etiquetas UI compartidas, botones, estados
  auth: { // Login, roles, credenciales
  nav: { // Items del menu de navegacion
  hierarchy: { // Terminos de jerarquia de equipos
  criticality: { // Terminos de evaluacion de riesgo
  fmea: { // Terminos de analisis de modos de falla
  tasks: { // Gestion de tareas
  workPackages: { // Operaciones de paquetes de trabajo
  analytics: { // Etiquetas y metricas de KPI
  reports: { // Tipos de reporte y exportacion
  planner: { // Interfaz del planificador IA + respuestas
  strategy: { // Etiquetas de estrategia RCM
  admin: { // Panel de administracion
  // ... mas modulos
}
```

10.4 Soporte RTL

El layout en árabe aplica automáticamente `dir="rtl"` al elemento raíz, invirtiendo flexbox, márgenes, padding y alineación de texto.

11. Integracion SAP

11.1 Vision General

La plataforma genera paquetes de datos compatibles con SAP para carga en el modulo SAP PM (Plant Maintenance). Durante el desarrollo, un servicio mock SAP proporciona datos de prueba realistas.

11.2 Datos Mock (`sap\_mock/data/`)

Archivo	Transaccion SAP	Contenido
work_orders.json	IW31/IW32	Ordenes PM historicas
functional_locations.json	IL01	Ubicaciones tecnicas
materials_bom.json	CS01	Lista de materiales
equipment_master.json	IE01	Registros maestros de equipos
maintenance_plans.json	IP10	Planes de mantenimiento predefinidos

11.3 Flujo de Carga SAP

1. Crear Paquete de Trabajo (agrupar tareas)
2. Aprobar Paquete de Trabajo
3. Generar Paquete de Carga SAP
4. Revisar en pagina de Revision SAP
5. Aprobar para transferencia SAP
6. Exportar en formato compatible SAP

11.4 Mapeo de Campos SAP

Campo OCP	-> Campo SAP
work_order_id	-> AUFNR (Numero de Orden)
equipment_tag	-> EQUNR (Numero de Equipo)
task_type (PM01-03)	-> AUART (Tipo de Orden)
priority	-> PRIOK (Prioridad)
description	-> KTEXT (Texto Breve)
planned_hours	-> ARBEI (Trabajo Planificado)

12. Docker y Despliegue

12.1 Servicios (docker-compose.yml)

```
services:
  ocp-backend:      # FastAPI + Gunicorn (4 workers)
    puertos: 8000
    volúmenes: ocp_db_data, ocp_sap_data
    healthcheck: HTTP GET /health

  ocp-frontend:     # Build React servido por Nginx
    puertos: 5173
    depende_de: ocp-backend (healthy)

  ocp-nginx:        # Proxy inverso
    puertos: 8080 (HTTP), 8443 (HTTPS)
    config: nginx.conf
```

12.2 Proceso de Build

Backend (Dockerfile):

```
Etapa 1: Python 3.11-slim -> Instalar dependencias
Etapa 2: Copiar paquetes -> Crear usuario no-root -> Ejecutar Gunicorn
```

Frontend (frontend/Dockerfile):

```
Etapa 1: Node 20-alpine -> npm ci -> npm run build
Etapa 2: Nginx Alpine -> Copiar dist/ -> Enrutamiento fallback SPA
```

12.3 Comandos Docker

```
# Construir e iniciar todos los servicios
docker-compose up -d --build

# Ver logs
docker-compose logs -f ocp-backend

# Detener todos los servicios
docker-compose down

# Reconstruir un solo servicio
docker-compose up -d --build ocp-frontend
```

12.4 Configuración Nginx

- Rate limiting: 30 req/s general, 2 req/s endpoints admin
- Headers de seguridad: X-Frame-Options, X-Content-Type-Options, X-XSS-Protection
- Proxy pass: /api y /health al backend, todo lo demás al frontend
- SSL listo: Configuración HTTPS comentada disponible para producción

13. Modulos Funcionales

Modulo 1: Jerarquia de Activos y Criticidad

Paginas: Hierarchy, Criticality

- Construir arbol de equipos desde datos maestros SAP o entrada manual
- Jerarquia de 6 niveles: Planta -> Area -> Sistema -> Equipo -> Sub-Ensamble -> Item Mantenible
- Evaluacion de matriz de riesgo con clases de riesgo sugeridas por IA
- Criterios: Seguridad, Medio Ambiente, Produccion, Calidad, Costo de Mantenimiento
- Salida: Clasificacion de riesgo (ALTO / MEDIO / BAJO)

Modulo 2: FMEA / FMECA y Estrategia RCM

Paginas: FMEA, FMECA, Strategy

- Definir funciones del equipo y fallas funcionales
- Crear modos de falla con mecanismos y causas estandarizados
- Calculo RPN (Numero de Prioridad de Riesgo): Severidad x Ocurrencia x Deteccion
- Logica de Decision RCM:
 - CBM · Mantenimiento Basado en Condicion (predictivo)
 - TBM · Mantenimiento Basado en Tiempo (preventivo)
 - FTM · Tarea de Busqueda de Falla (fallas ocultas)
 - RTF · Ejecutar Hasta la Falla (sin accion, aceptar riesgo)
- Generar tareas de mantenimiento desde decisiones RCM

Modulo 3: Operaciones y Planificacion

Paginas: WorkRequests, WorkPackages, Backlog, Scheduling, Planning, Planner, FieldCapture

- Tecnicos de campo capturan datos via formularios moviles
- Solicitudes de trabajo con clasificacion de prioridad asistida por IA
- Deteccion de duplicados para solicitudes similares
- Gestion de backlog con analisis de envejecimiento
- Agrupacion de tareas en paquetes de trabajo por area/ventana de parada
- Programacion con visualizacion GANTT
- Interfaz de chat del Planificador IA con recomendaciones contextuales

Modulo 4: Analitica, Reportes y Mejora Continua

Paginas: Analytics, ExecutiveDashboard, Reports, Reliability, RCA, DefectElimination, SAPReview

- KPIs: MTBF, MTTR, Disponibilidad, OEE, Adherencia al Programa
 - Analisis Weibull: Ajustar distribuciones de falla, predecir proxima falla
 - Puntuaciones de Salud: Evaluacion multidimensional de salud del activo
 - Reportes: Semanales, mensuales, trimestrales · exportables a Excel/CSV
 - RCA: Framework 5W2H (Quien, Que, Cuando, Donde, Por que, Como, Cuanto)
 - Eliminacion de Defectos: Seguimiento de fallas recurrentes y acciones correctivas
 - Revision SAP: Auditar paquetes de carga generados antes de la transferencia
-

14. Flujos de Trabajo Clave

14.1 Evaluacion de Criticidad

Seleccionar Equipo -> Llenar Criterios de Riesgo -> IA Sugiere Clase de Riesgo
-> Usuario Aprueba/Modifica -> Guardar Evaluacion -> Alimentar FMEA

14.2 FMEA a Paquete de Trabajo

Definir Funciones -> Identificar Fallas Funcionales -> Crear Modos de Falla
-> Aplicar Decision RCM -> Generar Tareas -> Agrupar en Paquetes de Trabajo
-> Aprobar -> Exportar a SAP

14.3 Ciclo de Vida de Solicitud de Trabajo

Captura en Campo -> Crear Solicitud de Trabajo -> Verificar Duplicados
-> Clasificacion de Prioridad IA -> Validar -> Agregar al Backlog
-> Programar -> Ejecutar -> Cerrar (soporte OCR)

14.4 Generacion y Exportacion de Reportes

Seleccionar Tipo de Reporte -> Configurar Parametros -> Generar (API o Mock)
-> Previsualizar Resultados -> Exportar Excel/CSV -> Descargar

15. Seguridad

Característica	Implementacion
Autenticacion	Tokens JWT (HS256), acceso 4 horas / refresco 7 dias
Almacenamiento de Contraseñas	Hash bcrypt con sal
CORS	Lista blanca de orígenes desde entorno
API Key	Aplicacion opcional de X-API-Key en mutaciones
Rate Limiting	Nginx: 30 req/s general, 2 req/s admin
Headers de Seguridad	X-Frame-Options: DENY, X-Content-Type-Options: nosniff
HTTPS	Nginx SSL/TLS 1.2+ listo (configuracion produccion provista)
Acceso por Roles	Guardias de ruta frontend + middleware backend
Contenedor No-Root	Backend ejecuta como appuser (UID 1000)
Proteccion SQL Injection	Consultas parametrizadas ORM SQLAlchemy
Proteccion CSRF	Token Bearer JWT (no basado en cookies)

16. Guia de Desarrollo Local

16.1 Prerequisitos

- Python 3.11+
- Node.js 20+
- npm 9+
- Git

16.2 Configuracion del Backend

```
# Clonar y navegar
cd ASSET-MANAGEMENT-SOFTWARE-master

# Crear entorno virtual
python -m venv .venv
source .venv/bin/activate      # Linux/Mac
.venv\Scripts\activate        # Windows

# Instalar dependencias
pip install -r requirements.txt

# Crear .env (copiar desde ejemplo)
cp .env.example .env

# Iniciar servidor backend
PYTHONPATH=. python -m uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
```

16.3 Configuracion del Frontend

```
cd frontend

# Instalar dependencias
npm install

# Iniciar servidor de desarrollo
npm run dev
```

16.4 Puntos de Acceso (Desarrollo)

URL	Servicio
http://localhost:5173	Frontend (React)
http://localhost:8000	API Backend
http://localhost:8000/docs	Documentacion API Swagger
http://localhost:8000/health	Health Check

16.5 Sembrar Base de Datos

```
# Via API (requiere backend corriendo)
curl -X POST http://localhost:8000/api/v1/admin/seed

# O via Python
PYTHONPATH=. python -c "from api.seed import seed_all; seed_all()"
```

17. Checklist de Produccion

- ☐ Cambiar JWT_SECRET_KEY a un valor aleatorio fuerte
 - ☐ Cambiar DATABASE_URL a PostgreSQL
 - ☐ Habilitar HTTPS en nginx.conf (descomentar bloque SSL)
 - ☐ Generar certificados SSL (certs/cert.pem, certs/key.pem)
 - ☐ Configurar API_KEY y ADMIN_API_KEY para proteccion de API
 - ☐ Configurar CORS_ORIGINS solo con dominios de produccion
 - ☐ Configurar ANTHROPIC_API_KEY para funciones de IA
 - ☐ Establecer DEBUG=false y LOG_LEVEL=WARNING
 - ☐ Revisar rate limiting en nginx.conf
 - ☐ Configurar montajes de volumen para datos persistentes
 - ☐ Configurar monitoreo y agregacion de logs
 - ☐ Ejecutar docker-compose up -d --build
-

18. Variables de Entorno

Variable	Valor por Defecto	Descripcion
DATABASE_URL	sqlite:///./ocp_maintenance.db	Cadena de conexion a la base de datos
SAP MOCK_DIR	sap_mock/data	Ruta a datos mock SAP
JWT_SECRET_KEY	ocp-maintenance-secret-key-...	Secreto de firma JWT
JWT_ALGORITHM	HS256	Algoritmo JWT
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	1440	TTL del token de acceso
JWT_REFRESH_TOKEN_EXPIRE_DAYS	7	TTL del token de refresco
CORS_ORIGINS	http://localhost:5173,...	Origenes CORS permitidos
API_KEY	_(vacio)_	API key para proteccion de mutaciones
ADMIN_API_KEY	_(vacio)_	API key de administrador
ANTHROPIC_API_KEY	_(vacio)_	API key de Claude AI
GOOGLE_AI_API_KEY	_(vacio)_	API key de Google AI
DEBUG	false	Habilitar logging de depuracion
LOG_LEVEL	INFO	Nivel de logging Python
BACKEND_PORT	8000	Puerto backend Docker
FRONTEND_PORT	5173	Puerto frontend Docker
NGINX_PORT	8080	Puerto HTTP Nginx Docker
NGINX_SSL_PORT	8443	Puerto HTTPS Nginx Docker

Estadísticas del Proyecto

Metrica	Cantidad
Routers API	18
Endpoints API	100+
Modelos de Base de Datos	25+
Servicios Backend	23
Paginas React	23
Componentes UI	20+
Esquemas Pydantic	50+
Funciones Cliente API	50+
Idiomas Soportados	3 (EN, ES, AR)
Servicios Docker	3
Dependencias Python	27
Dependencias npm	30+

OCP Maintenance AI Platform · Construido para la confiabilidad, diseñado para escalar.

OCP Maintenance AI Platform

Documentacion Tecnica Completa

Evaluado por Value Strategy Consulting

Fin del documento

Este documento contiene informacion tecnica sobre la plataforma OCP Maintenance AI. Su distribucion esta restringida al equipo de desarrollo y direccion.